Gestion de l'état et des données en SwiftUI

420-249-AH - Programmation mobile iOS

1. Qu'est-ce que l'état ?

L'état représente les données qui peuvent changer pendant l'utilisation de l'application.

Exemples :

- Nom saisi
- Compteur
- Bouton activé ou non
- Utilisateur connecté
- Élément sélectionné
- Liste de données

Lorsque l'état change, SwiftUI peut automatiquement mettre à jour l'interface.

2. Interface déclarative

Avec SwiftUI, on décrit l'interface en fonction de l'état actuel.

```
if estConnecte {
    Text("Bienvenue")
} else {
    Text("Veuillez vous connecter")
}
```

Si estConnecte change, SwiftUI recalcule automatiquement la vue.

3. @State

@State permet à une vue de conserver une donnée locale qui peut changer.

```
struct CompteurView: View {
    @State private var compteur = 0

    var body: some View {
        VStack {
            Text("Compteur : \(compteur)")
            Button("Ajouter") {
                compteur += 1
            }
        }
    }
}
```

Lorsque compteur += 1 s'exécute, SwiftUI actualise automatiquement Text("Compteur : \(compteur)").

4. Quand utiliser @State ?

Pour une donnée :

- Locale à la vue
- Modifiable
- Utilisée pour mettre à jour l'interface

Exemples :

```
@State private var nom = ""
@State private var notifications = true
@State private var compteur = 0
@State private var estConnecte = false
```

5. Donnée simple avec let

Si une valeur ne doit pas être modifiée par la vue :

```
struct ProfilView: View {
    let nom: String

    var body: some View {
        Text(nom)
    }
}
```

Utilisation :

```
ProfilView(nom: "Alex")
```

Ici, il n'est pas nécessaire d'utiliser @State.

6. @State avec un champ de saisie

```
struct FormulaireView: View {
    @State private var nom = ""

    var body: some View {
        VStack {
            TextField(
                "Nom",
                text: $nom
            )
            Text("Bonjour \(nom)")
        }
        .padding()
    }
}
```

Lorsque l'utilisateur modifie le champ :

```
TextField
   ↓
nom change
   ↓
Text est actualisé
```

7. Que signifie $?

Avec :

```
@State private var nom = ""
```

on utilise nom pour lire la valeur :

```
Text(nom)
```

Mais $nom permet à un contrôle de modifier cette valeur :

```
TextField(
    "Nom",
    text: $nom
)
```

nom représente la valeur elle-même, alors que $nom représente une liaison vers cette valeur. On appelle cette liaison un Binding.

8. @Binding

@Binding permet à une vue enfant de modifier une donnée appartenant à une autre vue.

Exemple :

```
struct InterrupteurView: View {
    @Binding var actif: Bool

    var body: some View {
        Toggle(
            "Actif",
            isOn: $actif
        )
    }
}
```

9. Pourquoi @Binding ?

Sans @Binding, une vue enfant recevrait simplement une copie de la valeur.

Avec @Binding, elle peut modifier directement la donnée gérée par la vue parent.

```
Parent
  @State
     ↓
  Binding
     ↓
Enfant
  @Binding
```

10. Exemple avec un bouton enfant

```swift
struct BoutonCompteur: View {
    @Binding var compteur: Int

    var body: some View {
        Button("Ajouter") {
            compteur += 1
        }
    }
}
```

Vue parent :

```swift
struct ContentView: View {
    @State private var compteur = 0

    var body: some View {
        VStack {
            Text("\(compteur)")
            BoutonCompteur(
                compteur: $compteur
            )
        }
    }
}
```

Le bouton, même défini dans une vue séparée, modifie directement le compteur du parent.

11. État et affichage conditionnel

```swift
@State private var estConnecte = false
```

Puis :

```swift
if estConnecte {
    Text("Utilisateur connecté")
} else {
    Text("Non connecté")
}
```

Bouton :

```
Button("Changer l'état") {
    estConnecte.toggle()
}
```

12. toggle()

Pour un Bool, on peut utiliser :

```
estConnecte.toggle()
```

au lieu de :

```
estConnecte = !estConnecte
```

13. État avec une collection

```
@State private var noms = [
    "Alex",
    "Marie"
]
```

On peut modifier la collection :

```
Button("Ajouter") {
    noms.append("Sam")
}
```

L'interface qui utilise cette collection sera automatiquement mise à jour.

14. Exemple avec ForEach

```
struct ListeView: View {
    @State private var noms = [
        "Alex",
        "Marie",
        "Sam"
    ]

    var body: some View {
        VStack {
            ForEach(noms, id: \.self) { nom in
                Text(nom)
            }
            Button("Ajouter") {
                noms.append("Nouvelle personne")
            }
        }
    }
}
```

15. Données partagées plus complexes

Lorsque plusieurs vues doivent travailler avec un même modèle, @State et @Binding peuvent devenir insuffisants.

Swift fournit notamment le système d'observation moderne :

```
@Observable
```

Exemple :

```
import Observation

@Observable
class CompteurModel {
    var valeur = 0
}
```

Puis :

```
struct ContentView: View {
    @State private var model = CompteurModel()

    var body: some View {
        Button("Valeur : \(model.valeur)") {
            model.valeur += 1
        }
    }
}
```

Nous approfondirons ce principe dans une présentation dédiée au patron MVVM en SwiftUI.

16. Ancienne approche

Dans de nombreux projets SwiftUI existants, vous pouvez aussi rencontrer :

```
ObservableObject
@Published
@StateObject
@ObservedObject
```

Exemple :

```
class CompteurModel: ObservableObject {
    @Published var valeur = 0
}
```

Cette approche reste importante à reconnaître, mais pour de nouveaux exemples nous pouvons privilégier le système moderne d'observation.

17. Exercice — Paramètres utilisateur

1. Écran de paramètres

Créer :

```
SettingsView
```

avec :

```
@State private var nom = ""
@State private var notifications = true
@State private var modeSombre = false
```

Afficher :

- Un TextField pour le nom
- Un Toggle pour les notifications
- Un Toggle pour le mode sombre

Afficher ensuite un résumé :

```
Console
Nom : Alex
Notifications : activées
Mode sombre : désactivé
```

2. Extraire le Toggle des notifications

Créer une vue :

```
NotificationSettingView
```

qui reçoit :

```
@Binding var notifications: Bool
```

Déplacer le Toggle des notifications dans cette vue. La modification effectuée dans la vue enfant doit immédiatement modifier la valeur dans la vue parent.